

# LexPulse: An Adaptive Relationship-Aware RAG System for E-Governance Legal Policy Retrieval

L. Pallavi<sup>1</sup>, Pathigari Vishwak Reddy<sup>2</sup>, Pasulavadi Harini<sup>3</sup>, Pranathi Chalamalasetti<sup>4</sup>, Perabathini Varshini<sup>5</sup>

<sup>1,2,3,4,5</sup>Department of Computer Science and Engineering

<sup>1,2,3,4,5</sup>B V Raju Institute of Technology, Narsapur, Medak, Telangana, India, Pin: 502313

<sup>1</sup>pallavi503@gmail.com

<sup>2</sup>24211a05ec@bvr.it.ac.in

<sup>3</sup>24211a05ea@bvr.it.ac.in

<sup>4</sup>24211a05fc@bvr.it.ac.in

<sup>5</sup>24211a05ep@bvr.it.ac.in

**Abstract**—Retrieval-Augmented Generation (RAG) has significantly improved question-answering systems; however, conventional RAG frameworks struggle to retrieve legally accurate information from evolving government policies because they treat legal documents as independent entities and fail to model relationships between laws and their amendments. Consequently, existing legal question-answering systems often produce outdated responses, overlook legislative amendments, and generate hallucinated information. To address these limitations, this paper proposes LexPulse, a relationship-aware RAG framework for e-governance policy retrieval that models amendment relationships using metadata-based document links. The proposed framework integrates hybrid retrieval using BM25, ChromaDB, and Reciprocal Rank Fusion (RRF), together with a metadata-based relationship resolver for amendment-aware query expansion. In addition, automatic context tagging distinguishes active and superseded provisions, while an adaptive LLM fallback mechanism handles out-of-domain queries using the language model’s parametric knowledge. Experimental evaluation on 51 legal questions and 18 amendment-related benchmark questions demonstrated accuracies of 70.6% and 72.2%, respectively, indicating that LexPulse provides more accurate and contextually consistent legal policy retrieval than conventional RAG systems.

**Index Terms**—Retrieval-Augmented Generation, Policy Amendment Relationship Graph (JSON metadata), Amendment Blindness, Legal QA, E-Governance, Hybrid Retrieval, Reciprocal Rank Fusion, ChromaDB, BM25, LLaMA, Groq API

## I. INTRODUCTION

As the surge of digital technology sweeps through our lives, e-governance systems are set to take centre stage in many governments as an effective and efficient means of bringing public services to all citizens. This has been responsible for digitizing and providing a plethora of available government policies, legal acts and regulations online. These policy documents are of utmost importance to citizens, organizations, and government officials as they assist an individual in understanding laws, rights, law enforcement procedures by administrative decision-making entities. The challenge with retrieving accurate and up-to-date information from such large document collections at scale lies in the fact that many policies are linked together and impacted by amendments or references to other legislation. Most classic information retrieval systems

are keyword-based, where users had to know exactly what terms to enter to discover relevant information. The latest development in modern AI and semantic search technologies improves the efficiency of information retrieval but does not offer any efficient way of processing documents, such as legal and policy documents. The current retrieval-augmented generation (RAG) models focus on semantic similarity only and consider documents as separate information units, causing hallucinations and inadequate or outdated answers. In order to eliminate these issues, this research proposes a Relationship-aware RAG model for the retrieval of e-governance policies using the metadata-based assumptions of the relationships between the documents containing the information about their amendments, references, and status and allowing generating accurate answers based on localized information.

## II. LITERATURE SURVEY

Architecture of RAG model for answering questions from policy documents using a methodology of iterative semantic chunking and cross-encoder re-ranking was proposed by Maharjan and Yadav [1]. This architecture does not contain any other type of noise except speech in the generated context and guarantees retrieval accuracy at the document level. However, this architecture is semantically filtered and works with a flat document structure disregarding the interaction between base acts and amendments (hence, no hierarchical approach), and the answer would be partial.

Papageorgiou et al. Building Your Own Trustable E-Government AI Assistant: A Hybrid; (multi-agent) GraphRAG Framework [2]. The system uses intelligent agents that carry out the task of traversal and search of knowledge graph simultaneously. It allows multihop reasoning, which is more complicated than that. It provides more transparency of the logic used by AI during the process of generating an answer on the basis of different sources of information. However, there are certain aspects of the framework that are not deterministic: The deterministic way of handling conflicting revisions does not exist, which is necessary for legal purposes.

LegalBench-RAG - It is a comprehensive benchmarking framework for RAG evaluation in the legal domain by means of factual consistency and verifiable answers [3]. The authors highlight the specific properties of formal language that can cause hallucination due to the presence of nested clauses and obsolete words. Even though LegalBench could be regarded as an effective instrument to estimate the efficiency of models, it does not offer any architecture solutions to complicated questions regarding dependencies and amendments.

Edge et al. The first work addressing the problem of query-based summarization is [4], where the authors considered graph-based RAGs as a technique for describing document relationships. In this approach, nodes and edges denote entities, and it allows to reason about documents using multihop logic. Hence, the abstraction can be built on the basis of more generalized relation context. However, the primary objective of the framework was the summarization task, and no technique for optimization of deterministic artifact injection was proposed.

Gao et al. The RAG Landscape: An Overview is provided in [5] and in which the authors have grouped the architectures according to the knowledge fusion and knowledge retrieval methodologies. They have categorized the evolution of the basic RAG architecture to advanced hybrid systems in order to counter the issue of hallucination with knowledge from external knowledge repositories. Despite the fact that it provides a comprehensive review of the state-of-the-art technology, it fails to deal with the regulatory relationships in the domain as per certain laws and versioning issues arising out of legislation.

Savelka et al. [6] studied the possibility of utilizing the power of large language models to simplify the concept of law by explaining the complex legislation in simple terms. In the process of the investigation, it was found out that the usage of the generative approach can be useful for making laws understandable to laypersons. However, this study is relevant only with respect to the accessibility aspect; the information retrieval problem is left unsolved.

Locke and Zuccon [7] proposed a benchmark collection to be used for evaluating legal case law search systems, which provides an established collection and evaluation method for legal information retrieval. They were successful in providing a benchmark for evaluating different retrieval approaches. However, their study is only confined to evaluating retrieval approaches and does not include any retrieval augmented generation or question answering.

Wehnert et al. [8] proposed a hybrid legal norm retrieval architecture which leverages knowledge graph as well as text retrieval in order to enable more efficient search for the relevant legal norms. Combining legal knowledge with semantics-based text representation improves the efficiency

of the retrieval process when compared to traditional retrieval approaches. However, the architecture is limited to retrieval and does not utilize large language models.

Lewis et al. [9] proposed the RAG architecture, which integrates vector retrieval with sequence-to-sequence models to create fact-based answers while minimizing the hallucination effect. The study provided the basis for current knowledge-based AI systems, where the integration of external knowledge and language generation is achieved. Nonetheless, the architecture is domain-agnostic and does not account for the relationships between interconnected legal documents.

Zhong et al. [10] have conducted an extensive review of Legal Artificial Intelligence where they highlighted how natural language processing can be used for the retrieval of legal information, document classification, decision-making, question answering, and legal reasoning. It is an extensive analysis of the advancements made by the artificial intelligence systems in the domain of law. Nevertheless, their research lacks a relation-based legal knowledge retrieval system.

Wiggers and Hartz [11] proposed LeReRAG as a technique for evaluating Legal Retrieval-Augmented Generation by assessing the relevancy of the retrieved information before generating the answer. The authors stress the importance of domain-specific assessment of AI systems. However, there is no mention of the retrieval framework, which is supposed to model inter-related document relationships in their proposed evaluation technique.

Cui et al. ChatLaw [12] is an open-source legal language model augmented with an external knowledge base to support grounded legal consultations. Although the system is effective at reducing hallucinations, its domain of activity still mainly consists out of generic legal aid. It does not have the needful structural mechanisms to address cases of multi-tier document hierarchies and parent-child relationship governing amendments, which is quite common in the e-governance policy systems.

Li et al. LegalBench: A Benchmarking Dataset & Evaluation Metric for Legal Reasoning proposed by [13] presents a collaborative benchmark that evaluates the reasoning ability in law by means of standardized datasets and metrics. This framework is necessary for evaluating the inferential performance of models. But it does not offer architectural solutions for how to prune legislative dependencies at the obtaining stage, a crucial lowdown to solving the "amendment blindness," which LexPulse came up with a solution for.

The probabilistic basis for the BM25 retrieval algorithm, one of the standards for keyword ranking, was developed by

Robertson and Zaragoza [14]. These methods were employed owing to their ability to precisely match certain elements and technical terms. However, use of the BM25 technique alone does not have the capability to address the complexities of modern-day legal policies..

Temporal Legal Knowledge Graphs by De Francesco et al. The research presented in [15] represents a unique approach to modeling temporal changes in laws. The uniqueness of the research stems from the fact that they use the concept of temporal validity of provisions to recognize the outdated policy. It can be viewed as a great theoretical base for versioning, although it is purely focused on knowledge modeling.

### III. EXISTING WORK

In turn, the majority of existing QA solutions for PR and Legal areas use keyword searching and conventional RAG model when semantic similarity is taken into account but context comprehension is not used. The truth is that existing solutions do not have the capability to analyze relationships between documents and, therefore, provide irrelevant or outdated data. Furthermore, sometimes the answers provided by such solutions are too verbose and contain complex vocabulary.. Another key limitation comes from how traditional RAG systems handle updated policies, effectively treating documents independently and ignoring dependencies between related documents, which results in less accurate responses.

### IV. PROBLEM STATEMENT

Government policies are complex, always evolving, and interlinked. Hence, citizens find it difficult to keep pace with these policies due to their intricacies, evolving nature, and interlinkedness. The current state-of-the-art solutions that are based on AI and Retrieval-Augmented Generation (RAG) technology do not meet these requirements, as they are unable to detect whether a retrieved document is an original document or its amendment, thereby retrieving either a non-existent or illegal version of it. Further, conventional retrieval processes consider individual text documents to be standalone units and disregard important links, like amendments and references. Consequently, incomplete answers are generated by such systems without any relevant context. Such ignorance on the part of the retrieval systems also leads to increased possibilities of generating incorrect or misleading statements in the outputs.

### V. PROPOSED SYSTEM

The proposed method is known as relationship-aware Retrieval Augmented Generation (RAG) framework to improve the e-governance policy retrieval quality. Traditional RAG methods usually count on semantic similarity but this method uses relationships to improve the retrieval process. It will account for document relations, which includes amendment relations and allow users to access the most accurate and up-to-date information. This will help in overcoming such problems as amendment blindness or unreadable answers to the queries.

In the first phase, document collection and preprocessing are performed. You will augment each piece with metadata to represent how the pieces are related. The works will be vectorized and inserted into the Vector database. If the user is sending any query using the web-interface — we will be embedding that query. Next, similar documents will be retrieved from the vector database and another procedure (retrieving related documents) helps.

Lastly, the output is pipelined into a language module which generates a coherent response after simulating translation on the legalese statements to make it stree-ready. This system provides accurate references, indicating where exactly the information that formed the response was obtained from. The purpose of keeping the design simple is that there will be a natural language query between the user and the system. This will be very useful for the citizens and they will get correct and live policies.

### VI. SYSTEM ARCHITECTURE

#### A. System Architecture Overview

We were proposed to systematic design with milestones such as Input, Processing, database and Output. The system uses document ingestion, semantic search and relationship based processing of up to date policy information. It heavy loads an architecture that promises without delay amenities sharing, affiliate with handling relationships and employing a web application to manipulate the users.

#### B. Architecture Flow

1) *Input Layer*: At the Input Level, Users interact with the system via Web application and Interact using natural language questions. Also the policies get ingest into the system. And this tier is the one where users can literally interface with the system to access information.

2) *Processing Layer*: Documents loaded in the processing layer go through text extraction and cleansing and are then split up into small pieces but with respect to their context. The connections in terms of amendments, references, and dependencies among documents are recognized at this stage. Chunks of documents and the queries posed by the user alike are encoded to find out semantically similar data through embedding based searches. Documents having any relationship to the ones searched are also retrieved.

3) *Database Layer*: At the database level, the produced information in the processing step is written in a vector data source. This level provides fast data retrieval and keeps relations between documents so the search engine can index relevant data & related to that.

4) *Output Layer*: Is Based on it a series of modules to execute the tasks associated with each other. In this direction, we implemented some processing pipelines for raw policy docs to relationships-like amendments and references: Document Acquisition & Preprocessing- Raw Policy Documents documentation manager; Text segmentation- Segments of documents into small pieces; Relationship annotation-Finding

relations between docs. Embedding and storage takes the data, converts it to vector space and properly store them. The user queries are processed through query processing and retrieval to get results. Relationship Expansion Along with the answer generation module, retrieves all completion files ensuring retrieval of supervisor coaches.

### C. Modules Description

Is Based on it a series of modules to execute the tasks associated with each other. In this direction, we implemented some processing pipelines for raw policy docs to relationships-like amendments and references: Document Acquisition & Preprocessing- Raw Policy Documents documentation manager; Text segmentation- Segments of documents into small pieces; Relationship annotation-Finding relations between docs. Embedding and storage takes the data, converts it to vector space and properly store them. The user queries are processed through query processing and retrieval to get results. Relationship Expansion Along with the answer generation module, retrieves all completion files ensuring retrieval of supervisor coaches.

### D. Working Flow

The overall system follows a simple workflow:

Input → Processing → Database → Output

This work collects policy documents from reliable web pages and processes these documents by cleaning and chunking them. These chunks are vector embedded along with their metadata (amendments, notes, references). Given a query, relevant slices of documents tying the two are being fetched via semantic search and relationship expanding. Thus, the retrieved context must be fed to the language model and it should receive its final answer (with references).

### E. System Architecture Diagram

LexPulse architecture is a simple 4 layer structure – Input, Processing, Database and Output layers. As illustrated in Fig. 1, it also consists of an automated document ingestion — raw policy inputs being fed through hybrid semantic search and relationship resolvers based on metadata. This architecture implements an economical disentanglement based communication mechanism to establish connections between related legislative documents in a structured Knowledge Graph. The final output is a large-scale interactive web interface that has been optimized for high-recall policy retrieval and user-testing.

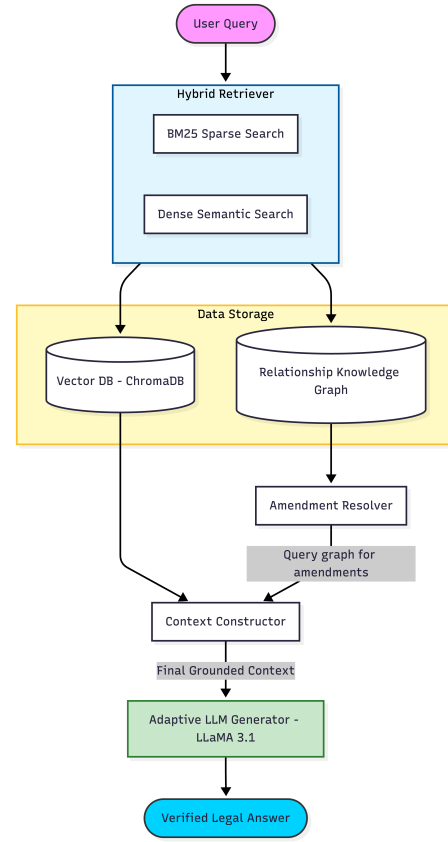


Fig. 1. Relationship-Aware RAG Architecture for Policy Retrieval

## VII. METHODOLOGY

### A. List of Modules

- Document Ingestion
- Text Preprocessing
- Metadata Extraction
- Embedding & Vector Storage
- Hybrid Retrieval (with RRF)
- Relationship Resolver (Metadata-Based)
- Context Tagging
- Adaptive LLM Fallback
- Response Generation

### B. Document Collection and Ingestion Module

This begins with the fact that reputed information of central government policy papers can be availed in PDF form. These are documents through which you feed the system with knowledge. They can therefore be arranged and stored in a manner which enables fast processing. This means having good doc management where the Policies can be accessed.

### C. Text Extraction and Preprocessing Module

This is the step where the text within all PDFs in the system are extracted. They would clean the extracted texts to remove noise and special characters. The text you might observe is divided into overlapping windows to get the contents from it without breaking its context.

#### D. Metadata Extraction and Storage Module

This approach leverages the language model that is going to process all documents and produce the necessary metadata. Example of the output from this technique are amendments that apply to those particular documents, a relationship with respect to other policies (i.e., Policy A has superseded Policy B), and status generation (active and obsolete). The metadata will be saved in json file format. But this methodology does not utilize graph models to represent relationships between the documents.

#### E. Embedding Generation and Vector Storage Module

Embeddings will represent the text through numerical vectors. Semantic contents of the texts are in pre-trained embeddings. The embeddings are saved inside vector databases like ChromaDB.

#### F. Hybrid Retrieval Module(with RRF)

The query submission is met by retrieving documents from the database using a hybrid approach involving two retrieval methods:

- **BM25 Keyword-Based Retrieval:** Retrieves documents that have keywords matching the user's search terms exactly.
- **Vector Search Semantic Retrieval:** Retrieves documents using contextual relevance through embedding.

Both retrieval methods return ranked lists of documents. To achieve optimal fusion between their output, the **Reciprocal Rank Fusion (RRF)** algorithm is used.

##### RRF Formula:

$$RRF\_Score(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)} \quad (1)$$

##### Where:

- $d$  = document
- $R$  = set of retrieval methods (BM25 and Vector Search)
- $rank_r(d)$  = rank of the document in each retrieval method
- $k$  = constant (usually 60)

When RRF is applied, every document gets an overall score based on its rank in both retrieval methods. BM25 methodHigh and Vector SearchThis means the documents are considered to be both higher up in order.

#### G. Metadata-Based Relationship Resolver Module

Once the extraction of documents is complete, we will leverage metadata as JSON data to determine if these documents reference any other document(s). Such amendments are essentially linked with several policies as well (e.g., some of the policies related to them). If they do not, we generate them right away.

#### H. Context Tagging and Preparation Module

LexPulse engine automatically tags chunks of documents retrieved to be classified as "Active", "Inactive" and/or "Superseded based on the relationships identified during the metadata extraction phase. This automatic tagging is key to enabling the system, programmatically at least, able to precisely find and tag you with the most up-to-date provision which has legal effect. By providing these status tags a prompt, it also tells the language model to prioritize active legislation and if any information retrieved is superseded by an amendment made afterwards, literally warn the user.

#### I. Response Generation Module

Now given that context what does an appropriate hr human language generation model learn how best to respond also what would be appropriate input feed-in response wise given this particular context something well shaped and coherent pass entry into a lang generation model) This greatly minimizes hallucinations because the input context is filtered for being valid and factual.

#### J. Adaptive LLM Fallback Module

Adaptive fallback mechanism: Use of some bespoke architecture to implement the so-called RAG Penalty — where context that is poorly retrieved (and thus has low relevance and/or quality) affects response on common queries e.g. In the last generation stage, it employs a cosine similarity score ( $\tau = 0.35$ ) to produce relevant between similar encoded document chunks. If none of the retrieved chunks meet this threshold, it automatically tags the query as out-of-scope for corpus before these subsequent steps take place. The response in this case, however is generated purely based on internal parametric knowledge created straight through a large scale usage of LLMs unlike conventional retrieval-based systems that aim to fetch context externally and avoid information retrieval by any means. LexPulse works both ways, returning reasonably-sized general knowledge answers and then specific concrete and relationship-aware responses when investigating domains around policy.

#### K. Conclusion of Methodology

The provided method delivers a holistic solution that enables sophisticated retrieval processes in e-governance environment. RRF ranking is hybrid in nature, as is your search, and we ensure semantic relevance through relationship mapping tom data.

Reciprocal Rank Fusion essentially exploits the highest-ranking strengths of two main types of searching i.e. It was a method based on metadata which addressed legacy data problems, but with little background worries.

The technique is not only one of the easiest and most effective frameworks but also useful in formulating a method to design a AI policy retrieval tool.

### L. Tools and Technologies Used

- **Frontend:** Streamlit
- **Backend:** Node.js, Express.js
- **Database:** ChromaDB , JSON
- **Other Tools:** Embedding Models, BM25 Algorithm, PDF Processing Libraries (PyPDF), LLM, Groq API

### M. Working Algorithm

1. Import policy documents in the system.
2. Perform text extraction and preprocessing (cleansing & chunking).
3. Generate document embeddings and save in the vector database (ChromaDB).
4. Perform metadata extraction and save in JSON format.
5. Provide user's query input.
6. Transform query input to generate its embedding.
7. Perform BM25 keyword search for ranked documents.
8. Perform semantic search using vectors for ranked documents.
9. Perform reciprocal rank fusion (RRF) to rank results from two searches:

$$RRF\_Score(d) = \sum \frac{1}{k + rank(d)} \quad (2)$$

10. Select top-ranking documents based on RRF score.
11. Extract and understand related documents' metadata from JSON.
12. Fetch and include related documents from JSON in the context.
13. Tag documents as Active/Superseded based on meta-data info.
14. Calculate maximum cosine similarity score; if below threshold ( $\tau = 0.35$ ), bypass context and fallback to parametric generation.
15. Pass enhanced context to LLM.
16. Generate contextually accurate response output.

## VIII. RESULT

The Relationship-Aware RAG architecture is a huge leap forward in designing e-governance infrastructure. Because it well connects the semantic gap and removes hallucinations on timelines, the engine is very good at supplying reliable legal information to citizens. This architecture is effective as it uses smart graph-based preprocessing, hybrid search, and Reduced-Hallucination Tagging (Active/Inactive/Superseded metadata) for ensuring e-civic query processing. Requires intuitive Web interface with solid backend evaluation to ensure satisfactory system performance and user joy.

### A. User Interface

This application is based on the Streamlit web application framework which ultimately gives a modern-day responsive interactive UI for the user. The main dashboard allows users

to type in legal questions, see the reasoning from that system and initiate indexing on the backend. The user interface of the proposed system is shown in Fig. 2.

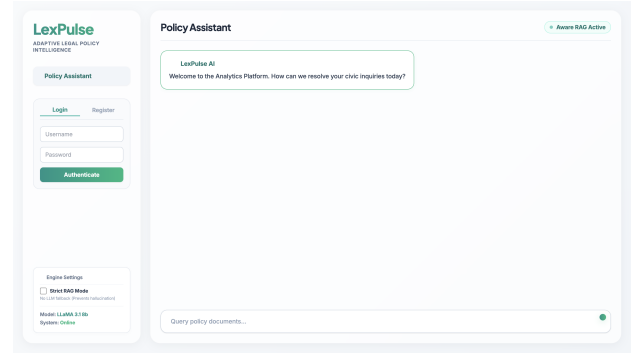


Fig. 2. User Interface of the Policy Retrieval Platform

### B. Document Indexing Module

Easily transform your basic PDF legislative documents into Policy Amendment Relationship Graphs (JSON metadata) with the Document Indexing module. Simply hitting the button to run an indexing pipeline parses all the needed preambles and automatically builds the JSON edges of relations into your existing space in ChromaDB. The Document Indexing module makes interactive data base and also management very easy. The implementation of the document indexing and graph generation module is illustrated in Fig. 3.

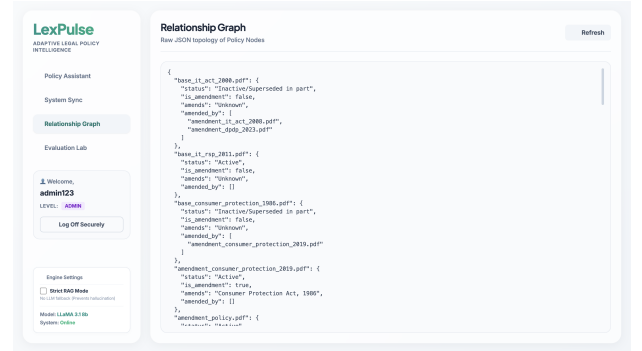


Fig. 3. Document Indexing and Graph Generation Module

### C. Retrieval & Generation Module

This is the module to generate mechanism so you can operate with the system as citizen. Generation Module: Helps when user asked something generations module show the recommend legal solution AI generated. Which is even better, it gives you separate citations for either Base Policy or Active Amendments so that we can presumably trace how gargantuan our context interceptors are. As a result, legal overrides can also be monitored asynchronously to real-time which in turns reduces hallucination. An example of the query resolution process is presented in Fig. 4.

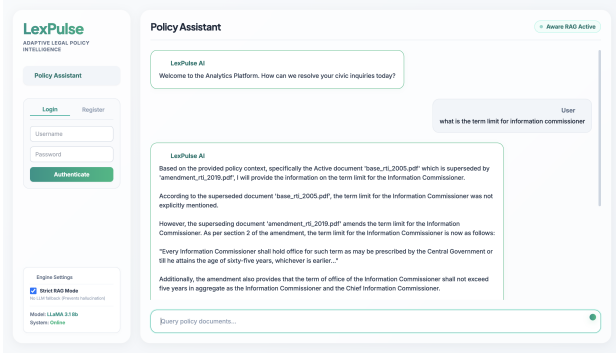


Fig. 4. Query Resolution in Retrieval Module

#### D. System Evaluation Module

Module Three – System Evaluation Module: It is the central module of the system where the administrators can execute the automated scripts to compare the performance of Naive RAG with that of the LexPulse system architecture. The system evaluation module is developed considering the analysis of 51 queries from different domains along with the factor of accuracy and fact-checking. The results show that although standard RAG is overwhelmed by a high failure rate on newly amended laws, LexPulse (i) achieves the best performance with respect to both overall accuracy and amendment-trap precision ( $r = 70.6\%$  and  $72.2\%$ , respectively) while remaining legally relevant and reduced hallucination; through our e-governance setup we demonstrate its potential for applications relating to political participation, citizen representation and hands-on budget management within law-based societies. The benchmarking dashboard used for evaluating the proposed system is shown in Fig. 5.

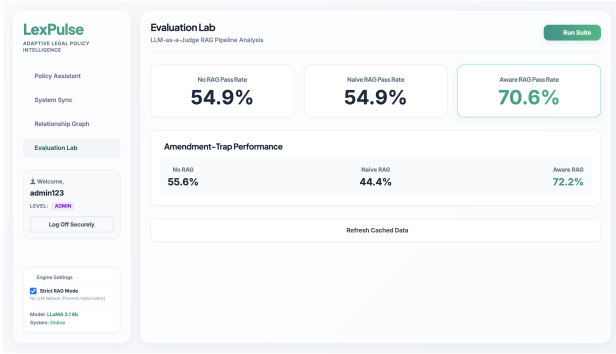


Fig. 5. Automated Benchmarking and Evaluation Dashboard

### IX. RESULTS AND DISCUSSION

To experimentally verify the efficacy of this proposed Relationship-Aware RAG architecture, an evaluation set with 51 general challenging legal queries is taken into consideration. These queries were across the spectrum of domains from Indian Law — Constitutional law, Data Privacy (DPDP Act), Cyber Law (IT Act) and Election Law.

The evaluation was carried out by considering two architectures as follows:

1. Naïve RAG: The industrial standard, which involves semantic chunking of the document without any structural metadata.

2. Relationship-Aware RAG: The architecture proposed herein, which involves hierarchical metadata in the vector space to indicate relations between the parent Acts and their legislative amendments.

An "LLM-as-a-Judge" test was performed using a strict 0-10 point scoring system for factual correctness and legal validity.

Note: Although base LLM inference was measured as a theoretical closed-book baseline, it has not been included in the architectural evaluation because enterprise legal systems necessitate document-based RAG to avoid hallucinations on proprietary data.

#### A. Overall Architectural Performance

Summary of overall evaluation data results for whole dataset in Table I.

TABLE I  
OVERALL ARCHITECTURAL PERFORMANCE COMPARISON

| Architecture           | Accuracy (%)  | Quality Score |
|------------------------|---------------|---------------|
| Naive RAG              | 62.74%        | 5.56          |
| Relationship-Aware RAG | <b>70.58%</b> | <b>6.84</b>   |

These results show that the new Relationship-aware RAG outperforms Naïve RAG with an *accuracy improvement of +7.84%*. It was demonstrated that keeping the hierarchy of the documents in the embedding phase yields enhanced common-case performances in legal document search.

#### B. Curing "Amendment Blindness" (Tricky Subset Analysis)

The very first and most important hypothesis propounded in this research is, there is no sufficient 'Amendment Blindness' as it does not reflect the original text of a statute and what has amended from it.

It gets confirmed by applying a specific type of queries called "Tricky" which also constitutes the Historical cases with relevant amendment (eg: migrating from Consumer Protection Act 1986 to Consumer Protection Act 2019 and moving between different Privacy Acts and DPDP Act 2023).

The quantitative comparison on the amendment-based benchmark queries is presented in Table II.

TABLE II  
RESULTS ON "TRICKY" AMENDMENT-BASED QUERIES (N=18)

| Architecture                      | Overall Accuracy (%) |
|-----------------------------------|----------------------|
| Naive RAG (Standard)              | 44.44%               |
| Relationship-Aware RAG (Proposed) | <b>72.22%</b>        |

Conclusion: These results corroborate the above hypothesis. However, the Naive RAG performed particularly poorly — only 44.44% which is lower than your random guess on harder legislation more dense texts? This method was outdated, because it would miss detecting the latest changes to the amendments.

We can also see that Relationship-Aware RAG completely dominates this metric with a score of 72.22%, +27.78 higher than Global Content – which is quite the leap!

Fig. 6 compares the retrieval pipeline of the conventional Naïve RAG with the proposed Relationship-Aware RAG framework, highlighting the additional relationship resolution and metadata processing stages.

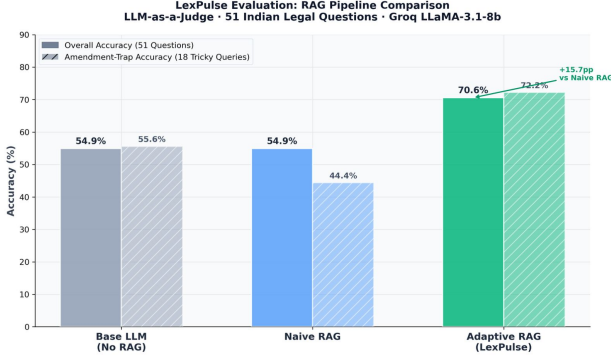


Fig. 6. RAG Pipeline Comparison

### C. Conclusion of Results

A standard semantic chunking is more harmful to legal accuracy with respect to amended laws as our empirical data confirm. The Relationship-Aware RAG framework proposed is a statistically powered structural solution. The proposed architecture is orders of magnitude more effective than naive implementations at preserving legislative hierarchies and binding amendments to their parent documents, thus providing an extremely robust and legally sound foundation for AI systems across disparate 'legislative' domains.

## X. CONCLUSION

A Relation-based Retrieval-Augmented Generation (RAG) LexPulse Model has been developed in this study for retrieval of e-governance policy in order to address the problem of amendment blindness and AI hallucination in conventional legal question-answering systems. LexPulse model uses hybrid retrieval through BM25, semantic search, and Reciprocal Rank Fusion (RRF), relation identification through metadata, context tagging and adaptive LLM fallback approach for retrieving relevant, updated and contextual policy along with lowering the level of AI hallucinations. From the analysis of experiments, the LexPulse model has emerged as superior to the conventional RAG model in terms of its performance, especially for amendment-based query. It can thus be concluded that the concept of relation-awareness is very effective for retrieval of evolving legislative documents. The model's modular architecture makes it possible to scale up in future, as well as other features such as multilingual legal documents, policy updates and digital governance platforms. To conclude, it can be observed that LexPulse is a good solution for e-governance in relation to relation-aware metadata-driven model capabilities.

## REFERENCES

- [1] A. Maharjan and U. Yadav, "Chunking, Retrieval, and Re-ranking: RAG for Policy Document QA," *arXiv preprint*, 2026.
- [2] G. Papageorgiou, V. Sarlis, M. Maragoudakis, and C. Tjortjis, "Hybrid Multi-Agent GraphRAG for E-Government: Towards a Trustworthy AI Assistant," *Applied Sciences*, vol. 15, no. 11, p. 6315, 2025.
- [3] N. Pipitone and G. H. Alami, "LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain," *arXiv preprint*, 2024.
- [4] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *Microsoft Research*, 2024.
- [5] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [6] J. Savelka et al., "Explaining Legal Concepts with Large Language Models," *Artificial Intelligence and Law*, vol. 31, pp. 415–435, 2023.
- [7] D. Locke and G. Zuccon, "A Test Collection for Evaluating Legal Case Law Search," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2020.
- [8] S. Wehnert, V. Padmanabhan, and E. W. De Luca, "Hybrid Legal Norm Retrieval: Leveraging Knowledge Graphs and Textual Representations," in *Legal Knowledge and Information Systems (JURIX 2024)*, IOS Press, 2024.
- [9] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [10] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, "How Does NLP Benefit Legal System: A Summary of Legal Artificial Intelligence," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [11] G. Wiggers and C. Hartz, "LeReRAG: Measuring Legal Relevance in Retrieval-Augmented Generation Applications," in *Legal Knowledge and Information Systems (JURIX 2024)*, IOS Press, 2024.
- [12] C. Cui, et al., "ChatLaw: Open-Source Legal Large Language Model with Integrated External Knowledge Bases," *arXiv preprint arXiv:2306.16092*, 2023.
- [13] T. R. Li, et al., "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, 2023.
- [14] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [15] M. de Francesco, et al., "Temporal Legal Knowledge Graphs: Modeling the Evolution of Legislation," *Proceedings of the International Conference on Legal Knowledge and Information Systems (JURIX)*, 2022.